

Name:	Muhammad Hasnat Fakhar
Class/Section:	BSCSev-F23-C
Reg ID:	232436
Subject:	Parallel and Distributing Computing
Task:	LAB TASK 07

LAB TASK 01

Code

```
1   using System;
2   using System.Collections.Concurrent;
3   using System.Diagnostics;
4   using System.Linq;
5   using System.Threading.Tasks;
6
    0 references
7   class Program
8   {
        7 references
9       const int N = 5_000_000;
        3 references
10      const int Reps = 3;
11
        15 references
12      static readonly int[] data = new int[N];
        2 references
13      static readonly object gate = new object();
        5 references
14      static int degree = Environment.ProcessorCount;
15
        1 reference | 3 references | 5 references
16      struct Partial { public long Sum; public int Max; }
        1 reference
17      delegate void Variant(out long sum, out int max);
18
        3 references
19      static ParallelOptions Options() =>
20          new ParallelOptions { MaxDegreeOfParallelism = degree };
21
        2 references
22      static void SerialVersion(out long sum, out int max)
23      {
24          long s = 0;
25          int m = int.MinValue;
26          for (int i = 0; i < N; i++)
27          {
28              s += data[i];
29              if (data[i] > m) m = data[i];
30          }
31          sum = s; max = m;
32      }
33
        1 reference
```

```
 7  class Program
34      static void RacyVersion(out long sum, out int max)
35      {
36          long s = 0;
37          int m = int.MinValue;
38          Parallel.For(0, N, Options(), i =>
39          {
40              s += data[i];                   // RACE — lost updates
41              if (data[i] > m) m = data[i];
42          });
43          sum = s; max = m;
44      }
45
            1 reference
46      static void LockEachVersion(out long sum, out int max)
47      {
48          long s = 0;
49          int m = int.MinValue;
50          Parallel.For(0, N, Options(), i =>
51          {
52              lock (gate)                     // ← lock har element pe
53              {
54                  s += data[i];
55                  if (data[i] > m) m = data[i];
56              }
57          });
58          sum = s; max = m;
59      }
60
            1 reference
61      static void LockPartialVersion(out long sum, out int max)
62      {
63          long s = 0;
64          int m = int.MinValue;
65          Parallel.ForEach(Partitioner.Create(0, N), Options(),
66              () => new Partial { Sum = 0, Max = int.MinValue },
67              (range, state, local) =>
68              {
69                  for (int i = range.Item1; i < range.Item2; i++)
70                  {
71                      local.Sum += data[i];
72                      if (data[i] > local.Max) local.Max = data[i];
73                  }
```

```
61      static void LockPartialVersion(out long sum, out int max)
74              return local;
75          },
76          local =>
77          {
78              lock (gate)                 // ← ek short lock per task
79              {
80                  s += local.Sum;
81                  if (local.Max > m) m = local.Max;
82              }
83          });
84      sum = s; max = m;
85  }
86
        1 reference
87  static void PlinqVersion(out long sum, out int max)
88  {
89      sum = data.AsParallel()
90                .WithDegreeOfParallelism(degree)
91                .Select(x => (long)x)
92                .Sum();
93      max = data.AsParallel()
94                .WithDegreeOfParallelism(degree)
95                .Max();
96  }
97
        0 references
98  static void Main(string[] args)
99  {
100     if (args.Length > 0) degree = int.Parse(args[0]);
101
102     string[] names = { "serial", "racy", "lock-each", "lock-partial", "plinq" };
103     Variant[] fns = { SerialVersion, RacyVersion, LockEachVersion,
104                       LockPartialVersion, PlinqVersion };
105
106     for (int i = 0; i < N; i++)
107         data[i] = (int)(((long)i * 7919) % 100003);
108
109     SerialVersion(out long refSum, out int refMax);
110     Console.WriteLine($"degree={degree} n={N} reps={Reps}");
```

```
112             for (int v = 0; v < fns.Length; v++)
113             {
114                 long sum = 0; int max = 0;
115                 double total = 0;
116                 for (int r = 0; r < Reps; r++)
117                 {
118                     Stopwatch sw = Stopwatch.StartNew();
119                     fns[v](out sum, out max);
120                     total += sw.Elapsed.TotalMilliseconds;
121                 }
122                 Console.WriteLine($"[{names[v],-13}] sum={sum} max={max} " +
123                                   $"sum_ok={(sum == refSum ? "YES" : "NO")} " +
124                                   $"max_ok={(max == refMax ? "YES" : "NO")} " +
125                                   $"avg_time={total / Reps:F2} ms");
126             }
127         }
128     }
```

Output

1. 4-Threads Run

```
PS E:\Lab-07> cd E:\Lab-07\Reduction
>> dotnet build -c Release
Restore complete (2.6s)
  Reduction net10.0 succeeded (37.7s) → bin\Release\net10.0\Reduction.dll

Build succeeded in 44.9s
PS E:\Lab-07\Reduction> dotnet run --no-build -c Release -- 4
degree=4 n=5000000 reps=3
[serial       ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=12.07 ms
[racy         ] sum=131861472032 max=100002 sum_ok=NO max_ok=YES avg_time=59.70 ms
[lock-each    ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=283.77 ms
[lock-partial ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=10.55 ms
[plinq        ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=144.93 ms
```

2. Scaling Runs (1, 2, 4, 8 threads)

```
PS E:\Lab-07\Reduction> foreach ($t in 1,2,4,8) {
>>     Write-Host "===== degree=$t ====="
>>     dotnet run --no-build -c Release -- $t
>> }
===== degree=1 =====
degree=1 n=5000000 reps=3
[serial       ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=9.99 ms
[racy         ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=83.79 ms
[lock-each    ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=205.55 ms
[lock-partial ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=21.70 ms
[plinq        ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=147.93 ms
===== degree=2 =====
degree=2 n=5000000 reps=3
[serial       ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=9.80 ms
[racy         ] sum=135210265259 max=100002 sum_ok=NO max_ok=YES avg_time=70.60 ms
[lock-each    ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=212.73 ms
[lock-partial ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=9.62 ms
[plinq        ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=124.25 ms
```

```
===== degree=4 =====
degree=4 n=5000000 reps=3
[serial       ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=10.95 ms
[racy         ] sum=165886938903 max=100002 sum_ok=NO max_ok=YES avg_time=60.56 ms
[lock-each    ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=238.62 ms
[lock-partial ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=8.30 ms
[plinq        ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=90.11 ms
===== degree=8 =====
degree=8 n=5000000 reps=3
[serial       ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=9.51 ms
[racy         ] sum=87452469792 max=100002 sum_ok=NO max_ok=YES avg_time=73.46 ms
[lock-each    ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=226.78 ms
[lock-partial ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=7.54 ms
[plinq        ] sum=250004979909 max=100002 sum_ok=YES max_ok=YES avg_time=71.70 ms
```

3. Racy Version — 3 Runs

```
PS E:\Lab-07\Reduction> 1..3 | ForEach-Object {
>>     Write-Host "===== Racy run $_ ====="
>>     dotnet run --no-build -c Release -- 4 | Select-String "racy"
>> }
===== Racy run 1 =====

[racy          ] sum=113863258339 max=100002 sum_ok=NO max_ok=YES
avg_time=48.19 ms
===== Racy run 2 =====
[racy          ] sum=129677271608 max=100002 sum_ok=NO max_ok=YES
avg_time=52.38 ms
===== Racy run 3 =====
[racy          ] sum=156825567682 max=100002 sum_ok=NO max_ok=YES
avg_time=58.50 ms
```

Result

Variants with 4 Threads

Variant	Sum correct? (Y/N)	Max correct? (Y/N)	Average time (ms)
serial	Y	Y	**12.07**
racy	**N**	Y	59.70
lock-each	Y	Y	**283.77**
lock-partial	Y	Y	**10.55**
plinq	Y	Y	**144.93**

Reference sum = 250,004,979,909 · **Reference max** = 100,002

Key observation: lock-partial (10.55 ms) actually **faster than serial** (12.07 ms) — real parallel speedup. But plinq (144.93 ms) is **12× slower than serial** — surprising! Reason: two separate PLINQ passes (.Sum() + .Max()), plus Select(x => (long)x) per element — overhead dominates on a 5M problem.

📊 **TABLE 2 — Scaling (Average Time in ms)**

Threads	lock-each	lock-partial	plinq	PLINQ speedup
1	205.55	21.70	147.93	**1.00×**
2	212.73	9.62	124.25	**1.19×**
4	238.62	8.30	90.11	**1.64×**
8	226.78	7.54	71.70	**2.06×**

Speedup formula: Speedup(t) = time(1 thread PLINQ) / time(t thread PLINQ)

Environment.ProcessorCount: 4

Discussion Questions

Q1: Why does the racy version give a wrong sum, and was the wrong value the same in every run? Why is the maximum often still correct?

The racy version gives a wrong sum because s += data[i] is a **non-atomic read-modify-write** (exactly the same lost-update problem as Lab 04). Multiple threads read the same value of s at the same time, add their own data[i], and write back — one thread's store overwrites the others, so many additions are lost.

The wrong value is **not the same in every run** — our three runs gave 113,863,258,339 / 129,677,271,608 / 156,825,567,682, and the different thread counts gave 131B, 165B, 87B, 135B. This is the classic signature of a race: nondeterministic, timing-dependent.

The maximum is often still correct because of how the update is written: if (data[i] > m) m = data[i]; — the shared m is only written when a strictly greater value is seen. Once some thread writes the true maximum (100,002), every later comparison data[i] > m fails, so no thread will overwrite it with a smaller value. The final m therefore converges to the correct maximum even though the access is still a data race (officially undefined behaviour; it just happens to give the right answer on this hardware).

Q2: Why is the lock-each version so much slower than the serial loop, and why does it get no faster (or even slower) with more threads?

The lock-each version is slow because **every single one of the 5 million elements** acquires and releases the same lock. Each lock operation is far more expensive than the

tiny amount of work being done (s += data[i]; if (...) m = ...). The serial loop performs the same work with **no lock at all**, so it wins easily — 12 ms vs 284 ms (≈ 24× slower).

It gets no faster with more threads because the lock **serializes** the loop. At any instant, only one thread can be inside the critical section; the others are blocked waiting. Adding more threads just adds more waiters queueing for the same lock — that's why we see 205 ms (1 thread), 213 ms (2), 239 ms (4), 227 ms (8). The wall time is essentially the serial execution time plus lock-acquisition overhead, not divided by the thread count. The lock-each version is the canonical example of "parallelism defeated by a fine-grained lock" — it turns a parallel loop back into a serial one.

Q3: Explain how the thread-local reduction works: what does each task start with, and when are the partial results combined? What are the identity values for + and for max?

The thread-local reduction works in three phases:

1. **Init phase** — each task creates its own private Partial { Sum = 0, Max = int.MinValue }. This is called once per task, not once per element. Each thread works on its own private memory — no shared state, no lock, no race.
2. **Body phase** — each task iterates over its assigned range, updating only its private local.Sum and local.Max. Because these are thread-local, all threads work in parallel at full speed. Returning local from the body passes the private state to the next call.
3. **Finally phase** — when a task finishes its range, localFinally runs **once** and acquires the shared lock briefly to merge the private result into the shared s and m. That's one lock per task (4 locks total for 4 threads), not one lock per element.

The **identity values** are the starting values that guarantee correctness for the reduce operation:

- **Sum (+)**: identity = **0** — adding 0 doesn't change the accumulator.
- **Max**: identity = **int.MinValue** — the smallest possible int, so any real value will overwrite it.

For a product the identity would be 1; for min it would be int.MaxValue. Getting the identity wrong (e.g., starting Max = 0) would give a wrong result if all data were negative.

Q4: Compare lock-partial with plinq. Do they do the same thing, and why is a library aggregation preferable to writing it by hand?

Yes — conceptually they do the **same thing**: each thread builds a private partial result and the partials are combined at the end. PLINQ is essentially the library's own implementation of the "thread-local state + merge" pattern, and lock-partial is our hand-rolled version of that same idea.

But in **our measurements** lock-partial (8–10 ms) is actually **faster than plinq** (72–148 ms). Why? Two reasons:

1. **One pass vs two passes** — lock-partial computes sum *and* max in a single pass over the data. The PLINQ version does **two completely separate queries** (.Sum() and .Max()), each with its own parallel partitioning, its own lambda dispatch, and its own reduction. That doubles the total work.
2. **Per-element lambda overhead** — Select(x => (long)x) invokes a delegate for every one of the 5 million elements; on a small problem this overhead dominates the arithmetic.

Library aggregation is still generally preferable in production because:

- **Correctness** — no risk of forgetting a lock, a private variable, or an identity value.
- **Maintainability** — one line of PLINQ vs ~20 lines of hand-written Parallel.ForEach with localInit/localFinally.
- **Portability** — PLINQ is tuned per-platform by the runtime team; the hand version is only as fast as you can make it.

But the **benchmark matters**: for very small problems the manual version can win, and this lab's measurements make that trade-off visible. Rule of thumb: **start with the library; only hand-optimise when measurements prove the library is too slow**.

Q5: The program uses integers. Why could a floating-point sum computed with a reduction differ slightly between runs or thread counts, even though the program is correct?

Floating-point addition is **not associative** in IEEE-754 arithmetic:

(a + b) + c ≠ a + (b + c) // in general

Each addition may **round** the result to fit the 52-bit mantissa of a double. Where the rounding happens depends on the **order of additions**, and the order depends on how the reduction groups the data.

With a thread-local reduction, each thread adds up its own chunk — the same list of numbers gets grouped into (t0-partial + t1-partial + t2-partial + ...) at the end. Different thread counts, different chunk sizes, or different scheduling produce **different groupings** → different rounding at different steps → a final answer that differs in the last few bits (e.g., 2.5000000001 vs 2.5000000002).

This is **not a bug** — the algorithm is still correct in the sense that it computes *a* sum of the input values. It just computes it with a different floating-point rounding path. Integer addition is **exactly associative** (no rounding), which is why all of our integer sums match the reference exactly. That's also why OpenMP has a reduction clause and why PLINQ's Sum() on double is documented as nondeterministic at the last bit.

LAB TASK 02

Code

```
using System;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;

13 references
class Policy
{
    2 references
    public string Name;
    3 references
    public string Kind;   // static, dynamic, guided or library
    9 references
    public int Chunk;
    8 references
    public Policy(string name, string kind, int chunk)
    {
        Name = name; Kind = kind; Chunk = chunk;
    }
}

0 references
class Program
{
    14 references
    const int N = 20_000;
    1 reference
    const int Reps = 3;
```

```
21      static int degree = 4;
        3 references
22      static bool uniform = false;
23
        4 references
24      static readonly long[] result   = new long[N];
        2 references
25      static readonly long[] expected = new long[N];
26
        4 references
27      static int nextIndex;                       // shared claim counter
        1 reference
28      static readonly object claimLock = new object();
29
        3 references
30      static long Work(int i)
31      {
32          int steps = uniform ? N / 2 : i;        // uneven: iteration i costs i s
33          long acc = i;
34          for (int j = 0; j < steps; j++)
35              acc = unchecked(acc * 31 + (j ^ i));
36          return acc;
37      }
38
        4 references
39      static void Process(int start, int end)
40      {
41          for (int i = start; i < end; i++)
42              result[i] = Work(i);
43      }
44
        1 reference
45      static void Worker(Policy p, int id)
46      {
47          switch (p.Kind)
48          {
49              case "static" when p.Chunk == 0:    // one equal block per thread
50              {
51                  int start = (int)((long)id * N / degree);
52                  int end   = (int)((long)(id + 1) * N / degree);
53                  Process(start, end);
54                  break;
55              }
56              case "static":                       // chunks dealt out round-robin
57                  for (int s = id * p.Chunk; s < N; s += degree * p.Chunk)
58                      Process(s, Math.Min(s + p.Chunk, N));
```

Debug Any CPU C# Workspace Dev Mode Live Share

```
59                  break;
60
61              case "dynamic":
62              {
63                  while (true)
64                  {
65                      int s = Interlocked.Add(ref nextIndex, p.Chunk) - p.Chunk;
66                      if (s >= N) break;
67                      Process(s, Math.Min(s + p.Chunk, N));
68                  }
69                  break;
70              }
71              case "guided":
72              {
73                  while (true)
74                  {
75                      int s, size;
76                      lock (claimLock)
77                      {
78                          s = nextIndex;
79                          int remaining = N - s;
80                          if (remaining <= 0) break;
81                          size = Math.Max(p.Chunk, remaining / (2 * degree));
82                          if (size > remaining) size = remaining;
83                          nextIndex = s + size;
84                      }
85                      Process(s, s + size);
86                  }
87                  break;
88              }
89          }
90      }
91
        1 reference
92      static void RunPolicy(Policy p, out double wall, out double bmax, out double bmin)
93      {
94          Array.Clear(result, 0, N);
95          nextIndex = 0;
96          double[] busy = new double[degree];
97          Stopwatch total = Stopwatch.StartNew();
98
99          if (p.Kind == "library")
100         {
101             ParallelOptions options = new ParallelOptions { MaxDegreeOfParallelism = degree };
```

```
102             Parallel.For(0, N, options, i => result[i] = Work(i));
103             bmax = bmin = -1;                       // per-thread times not available
104         }
105         else
106         {
107             Thread[] threads = new Thread[degree];
108             for (int t = 0; t < degree; t++)
109             {
110                 int id = t;
111                 threads[t] = new Thread(() =>
112                 {
113                     Stopwatch sw = Stopwatch.StartNew();
114                     Worker(p, id);
115                     busy[id] = sw.Elapsed.TotalMilliseconds;
116                 });
117                 threads[t].Start();
118             }
119             foreach (Thread th in threads) th.Join();
120 
121             bmax = busy[0]; bmin = busy[0];
122             foreach (double b in busy)
123             {
124                 if (b > bmax) bmax = b;
125                 if (b < bmin) bmin = b;
126             }
127         }
128         wall = total.Elapsed.TotalMilliseconds;
129     }
130 
        0 references
131     static void Main(string[] args)
132     {
133         foreach (string a in args)
134         {
135             if (a == "uniform") uniform = true;
136             else degree = int.Parse(a);
137         }
138 
139         Policy[] policies =
140         {
141             new Policy("static",       "static",  0),
142             new Policy("static,1",     "static",  1),
143             new Policy("static,64",    "static",  64),
144             new Policy("dynamic,1",    "dynamic", 1),
```

```
138
139         Policy[] policies =
140         {
141             new Policy("static",       "static",  0),
142             new Policy("static,1",     "static",  1),
143             new Policy("static,64",    "static",  64),
144             new Policy("dynamic,1",    "dynamic", 1),
145             new Policy("dynamic,64",   "dynamic", 64),
146             new Policy("guided,1",     "guided",  1),
147             new Policy("guided,64",    "guided",  64),
148             new Policy("Parallel.For", "library", 0)
149         };
150
151         Stopwatch s0 = Stopwatch.StartNew();
152         for (int i = 0; i < N; i++) expected[i] = Work(i);
153         s0.Stop();
154
155         string kind = uniform ? "uniform" : "uneven";
156         Console.WriteLine($"workload={kind} n={N} degree={degree} " +
157                           $"serial={s0.Elapsed.TotalMilliseconds:F1} (all times in ms)");
158
159         foreach (Policy p in policies)
160         {
161             double best = double.MaxValue, bmax = 0, bmin = 0;
162             bool ok = true;
163             for (int r = 0; r < Reps; r++)
164             {
165                 RunPolicy(p, out double w, out double mx, out double mn);
166                 if (!result.AsSpan().SequenceEqual(expected)) ok = false;
167                 if (w < best) { best = w; bmax = mx; bmin = mn; }
168             }
169             double ratio = bmin > 0.001 ? bmax / bmin : 0.0;
170             string busyText = bmax < 0
171                 ? "busy_max= n/a  busy_min= n/a  imbalance= n/a"
172                 : $"busy_max={bmax,7:F1} busy_min={bmin,7:F1} imbalance={ratio,5:F2}";
173             string okText = ok ? "YES" : "NO";
174             Console.WriteLine($"[{p.Name,-12}] wall={best,7:F1} {busyText} correct={okText}");
175         }
176     }
177 }
```

Output

```
>> # 6. Run 4 threads (uniform workload) — Task 4 ke liye save karo
>> dotnet run --no-build -c Release -- 4 uniform
Restore complete (0.7s)
  Task2-SchedulingBench net10.0 succeeded (0.3s) → bin\Release\net10.0\Task2-SchedulingBench.dll

Build succeeded in 1.8s
workload=uneven n=20000 degree=4 serial=271.0 (all times in ms)
[static      ] wall=  109.5 busy_max=  108.4 busy_min=   18.2 imbalance= 5.95 correct=YES
[static,1    ] wall=   80.9 busy_max=   76.6 busy_min=   73.3 imbalance= 1.05 correct=YES
[static,64   ] wall=   82.1 busy_max=   77.7 busy_min=   71.0 imbalance= 1.09 correct=YES
[dynamic,1   ] wall=   84.9 busy_max=   84.3 busy_min=   68.5 imbalance= 1.23 correct=YES
[dynamic,64  ] wall=   74.9 busy_max=   74.0 busy_min=   72.5 imbalance= 1.02 correct=YES
[guided,1    ] wall=  107.4 busy_max=  106.8 busy_min=  105.7 imbalance= 1.01 correct=YES
[guided,64   ] wall=   84.0 busy_max=   82.4 busy_min=   77.9 imbalance= 1.06 correct=YES
[Parallel.For] wall=   94.4 busy_max= n/a  busy_min= n/a  imbalance= n/a correct=YES
workload=uneven n=20000 degree=2 serial=357.3 (all times in ms)
[static      ] wall=  172.4 busy_max=  170.5 busy_min=   64.8 imbalance= 2.63 correct=YES
[static,1    ] wall=  126.7 busy_max=  124.4 busy_min=  119.0 imbalance= 1.05 correct=YES
[static,64   ] wall=  130.3 busy_max=  129.8 busy_min=  129.6 imbalance= 1.00 correct=YES
[dynamic,1   ] wall=  131.7 busy_max=  131.0 busy_min=  130.0 imbalance= 1.01 correct=YES
[dynamic,64  ] wall=  114.7 busy_max=  114.2 busy_min=  114.1 imbalance= 1.00 correct=YES
[guided,1    ] wall=  105.6 busy_max=  105.3 busy_min=  105.1 imbalance= 1.00 correct=YES
[guided,64   ] wall=  112.5 busy_max=  112.2 busy_min=  111.1 imbalance= 1.01 correct=YES
[Parallel.For] wall=   95.7 busy_max= n/a  busy_min= n/a  imbalance= n/a correct=YES
workload=uneven n=20000 degree=8 serial=270.2 (all times in ms)
[static      ] wall=  109.4 busy_max=   68.7 busy_min=    4.8 imbalance=14.28 correct=YES
[static,1    ] wall=   78.6 busy_max=   41.6 busy_min=   34.5 imbalance= 1.20 correct=YES
[static,64   ] wall=   82.0 busy_max=   68.6 busy_min=   34.7 imbalance= 1.98 correct=YES
[dynamic,1   ] wall=   74.4 busy_max=   72.8 busy_min=    0.0 imbalance= 0.00 correct=YES
[dynamic,64  ] wall=   78.6 busy_max=   76.8 busy_min=    0.0 imbalance= 0.00 correct=YES
[guided,1    ] wall=   81.2 busy_max=   79.2 busy_min=    0.0 imbalance= 0.00 correct=YES
[guided,64   ] wall=   81.2 busy_max=   80.1 busy_min=    0.0 imbalance= 0.00 correct=YES
[Parallel.For] wall=   81.4 busy_max= n/a  busy_min= n/a  imbalance= n/a correct=YES
workload=uniform n=20000 degree=4 serial=271.3 (all times in ms)
[static      ] wall=   92.0 busy_max=   91.2 busy_min=   80.6 imbalance= 1.13 correct=YES
[static,1    ] wall=   75.4 busy_max=   74.1 busy_min=   66.8 imbalance= 1.11 correct=YES
[static,64   ] wall=   73.2 busy_max=   72.9 busy_min=   68.7 imbalance= 1.06 correct=YES
[dynamic,1   ] wall=   76.6 busy_max=   76.2 busy_min=   69.6 imbalance= 1.09 correct=YES
[dynamic,64  ] wall=   74.7 busy_max=   74.3 busy_min=   68.1 imbalance= 1.09 correct=YES
[guided,1    ] wall=   71.4 busy_max=   70.7 busy_min=   69.1 imbalance= 1.02 correct=YES
[guided,64   ] wall=   82.3 busy_max=   81.7 busy_min=   80.9 imbalance= 1.01 correct=YES
[Parallel.For] wall=   71.8 busy_max= n/a  busy_min= n/a  imbalance= n/a correct=YES
PS E:\Lab-07\Task2-SchedulingBench>
```

Result

TABLE 1 — Uneven Workload with 4 Threads

Policy	Wall time (ms)	Busy max (ms)	Busy min (ms)	Imbalance	Correct?
static	109.5	108.4	18.2	**5.95**	Y
static,1	80.9	76.6	73.3	1.05	Y
static,64	82.1	77.7	71.0	1.09	Y
dynamic,1	84.9	84.3	68.5	1.23	Y
dynamic,64	**74.9**	74.0	72.5	1.02	Y
guided,1	107.4	106.8	105.7	1.01	Y
guided,64	84.0	82.4	77.9	1.06	Y
Parallel.For	94.4	n/a	n/a	n/a	Y

Environment.ProcessorCount: 4

Serial time (ms): 271.0

Fastest policy at 2 / 4 / 8 threads: Parallel.For (95.7 ms) / dynamic,64 (74.9 ms) / dynamic,1 (74.4 ms)

DISCUSSION ANSWERS

1. Why does the plain static policy show a large imbalance on this workload? Identify which thread (or worker) is busy longest and explain why.

The workload is triangular: iteration i costs i steps, so the **last iterations are the most expensive**. Under plain static (one contiguous block per thread), the last worker (Thread 3) receives the block [15000, 20000), which contains all the heaviest iterations. It runs for **108.4 ms**, while Thread 0 gets the cheap first block [0, 5000) and finishes in **18.2 ms**. Hence the imbalance ratio is **5.95×**. The wall time is set by the slowest thread, so the whole program waits for Thread 3.

2. Explain why static,1, the dynamic policies and the guided policies balance the work much better, and what each of them pays for it.

- static,1: deals chunks round-robin (Thread 0 gets 0, 4, 8; Thread 1 gets 1, 5, 9). Each thread sees a mix of cheap and expensive iterations. Balance improves to **1.05×**. **Cost:** computing a chunk start for every iteration (cheap, no atomic).
- dynamic,1 **/** dynamic,64: each worker claims the next chunk with Interlocked.Add when free. Fast workers grab more chunks. Balance is excellent (**1.02×**). **Cost:** atomic operations on a shared counter, which can be expensive at chunk size 1.
- guided,1 **/** guided,64: claim under a lock, starting with large chunks and shrinking toward the chunk size. Reduces the number of claims (less lock contention) while still balancing near the end. **Cost:** lock overhead, and balance isn't quite as tight as dynamic,1 here.

3. How did the results change with 2 and with 8 threads? When the number of threads is larger than the number of cores, what does the busy time of a thread really measure?

- **2 threads:** fewer threads compete, so plain static imbalance drops to **2.63×**. Dynamic/guided achieve near-perfect balance (**1.00×**). Wall times are higher because less parallelism is available.
- **8 threads:** plain static imbalance explodes to **14.28×** (max 68.7 ms, min 4.8 ms). However, dynamic/guided report **imbalance = 0.00**, which is a *measurement artefact*: busy_min is 0.0 because **8 threads on a 4-core machine** means some threads never actually run — the OS time-slices them. Their busy counter never starts, so the minimum becomes 0. **The busy time then measures CPU time (time actually running), not wall time**, making the imbalance metric meaningless when threads > cores.

4. What is the effect of the chunk size? Compare chunk size 1 with chunk size 64 for the dynamic policy, and name the trade-off.

With dynamic,1, each iteration is a separate claim — perfect load balancing but 20,000 atomic operations. With dynamic,64, there are only ~312 claims — much cheaper, but balance is slightly coarser. In our 4-thread run, dynamic,64 (**74.9 ms**) actually beat dynamic,1 (**84.9 ms**) because the atomics cost more than the imbalance saved. The trade-off: **smaller chunks = better balance, more overhead; larger chunks = less overhead, coarser balance**.

5. Why does guided start with large chunks and then shrink them? In what kind of workload would that be an advantage over dynamic,1?

Guided starts with a chunk of remaining / (2 * degree) and shrinks toward the minimum chunk size. The advantage:

- **Large chunks first** → few claims → low scheduling overhead when there's plenty of work.
- **Small chunks later** → fine-grained balance near the end, when only a few iterations remain

This is ideal for **workloads with a heavy tail** (like our triangular workload) or irregular workloads. dynamic,1 pays the atomic cost on every iteration; guided pays it fewer times while keeping good balance at the end.

LAB TASK 03

Code

Task3-FalseSharing > Program.cs > Program

```
using System;
using System.Diagnostics;
using System.Threading;

0 references
class Program
{
    7 references
    const long Iters = 50_000_000;
    1 reference
    const int CacheLine = 64;
    4 references
    const int Stride = CacheLine / sizeof(long); // 8 longs = 64 byte
    3 references
    static long[] adjacent;      // slots 8 bytes apart
    3 references
    static long[] padded;        // slots 64 bytes apart
    3 references
    static long[] localResult;   // written once per thread

    2 references
    static double Run(int variant, int threads)
    {
        Thread[] ts = new Thread[threads];
        Stopwatch sw = Stopwatch.StartNew();

        for (int t = 0; t < threads; t++)
        {
            int id = t;
            ts[t] = new Thread(() =>
            {
```

```
15      static double Run(int variant, int threads)
25                  if (variant == 0)
26                  {
27                      // Adjacent: 8 bytes apart (false sharing)
28                      for (long i = 0; i < Iters; i++)
29                          adjacent[id]++;
30                  }
31                  else if (variant == 1)
32                  {
33                      // Padded: 64 bytes apart (no false sharing)
34                      int idx = (id + 1) * Stride;
35                      for (long i = 0; i < Iters; i++)
36                          padded[idx]++;
37                  }
38                  else
39                  {
40                      // Local accumulate: private variable, write once
41                      long local = 0;
42                      for (long i = 0; i < Iters; i++)
43                          local++;
44                      localResult[id] = local;
45                  }
46              });
47              ts[t].Start();
48          }
49
50          foreach (Thread th in ts) th.Join();
51          sw.Stop();
52          return sw.Elapsed.TotalMilliseconds;
53      }
54
        0 references
55      static void Main(string[] args)
56      {
57          int threads = Math.Min(Environment.ProcessorCount, 8);
58          if (args.Length > 0) threads = int.Parse(args[0]);
59
60          adjacent    = new long[threads];
61          padded      = new long[(threads + 1) * Stride];
62          localResult = new long[threads];
63
64          Console.WriteLine($"threads={threads} iterations={Iters} per thread");
65          Console.WriteLine($"adjacent slots are {sizeof(long)} bytes apart; " +
66                            $"padded slots are {Stride * sizeof(long)} bytes apart");
```

```
60          adjacent    = new long[threads];
61          padded      = new long[(threads + 1) * Stride];
62          localResult = new long[threads];
63
64          Console.WriteLine($"threads={threads} iterations={Iters} per thread");
65          Console.WriteLine($"adjacent slots are {sizeof(long)} bytes apart; " +
66                            $"padded slots are {Stride * sizeof(long)} bytes apart");
67
68          string[] names = { "adjacent", "padded", "local-accumulate" };
69          double[] ms = new double[3];
70
71          // Run each variant twice, take the second run (first is warm-up)
72          for (int v = 0; v < 3; v++)
73          {
74              Run(v, threads);            // warm-up
75              ms[v] = Run(v, threads);    // actual
76          }
77
78          bool ok = true;
79          for (int t = 0; t < threads; t++)
80              if (adjacent[t] != Iters || padded[(t + 1) * Stride] != Iters ||
81                  localResult[t] != Iters)
82                  ok = false;
83
84          for (int v = 0; v < 3; v++)
85              Console.WriteLine($"[{names[v],-16}] time={ms[v],8:F2} ms " +
86                                $"slowdown vs padded={ms[v] / ms[1],5:F2}x");
87
88          Console.WriteLine($"counters correct={(ok ? "YES" : "NO")}");
89      }
90  }
```

Output

```
Build succeeded with 3 warning(s) in 6.0s
PS E:\Lab-07\Task3-FalseSharing> # 4 threads — 3 times (for first table)
>> Write-Host "===== Run 1 ====="; dotnet run --no-build -c Release -- 4
>> Write-Host "===== Run 2 ====="; dotnet run --no-build -c Release -- 4
>> Write-Host "===== Run 3 ====="; dotnet run --no-build -c Release -- 4
>>
>> # Scaling (for second table)
>> foreach ($t in 1,2,4,8) {
>>     Write-Host "===== threads=$t ====="
>>     dotnet run --no-build -c Release -- $t
>> }
===== Run 1 =====
threads=4 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=  301.59 ms slowdown vs padded= 1.18x
[padded          ] time=  255.48 ms slowdown vs padded= 1.00x
[local-accumulate] time=   33.96 ms slowdown vs padded= 0.13x
counters correct=NO
===== Run 2 =====
threads=4 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=  318.75 ms slowdown vs padded= 2.15x
[padded          ] time=  147.97 ms slowdown vs padded= 1.00x
[local-accumulate] time=   57.19 ms slowdown vs padded= 0.39x
counters correct=NO
===== Run 3 =====
threads=4 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=  330.43 ms slowdown vs padded= 1.44x
[padded          ] time=  228.91 ms slowdown vs padded= 1.00x
[local-accumulate] time=   73.04 ms slowdown vs padded= 0.32x
counters correct=NO
===== threads=1 =====
threads=1 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=   76.15 ms slowdown vs padded= 0.95x
[padded          ] time=   80.15 ms slowdown vs padded= 1.00x
[local-accumulate] time=   17.98 ms slowdown vs padded= 0.22x
counters correct=NO
===== threads=2 =====
threads=2 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=  155.21 ms slowdown vs padded= 1.94x
[padded          ] time=   79.95 ms slowdown vs padded= 1.00x
[local-accumulate] time=   29.73 ms slowdown vs padded= 0.37x
counters correct=NO
===== threads=4 =====
threads=4 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=  299.91 ms slowdown vs padded= 2.75x
[padded          ] time=  108.91 ms slowdown vs padded= 1.00x
[local-accumulate] time=   40.06 ms slowdown vs padded= 0.37x
counters correct=NO
===== threads=8 =====
threads=8 iterations=50000000 per thread
adjacent slots are 8 bytes apart; padded slots are 64 bytes apart
[adjacent        ] time=  603.43 ms slowdown vs padded= 2.34x
[padded          ] time=  257.97 ms slowdown vs padded= 1.00x
[local-accumulate] time=  179.96 ms slowdown vs padded= 0.70x
counters correct=NO
PS E:\Lab-07\Task3-FalseSharing>
```

Result

TABLE 1 — Three runs with 4 threads

Run	adjacent (ms)	padded (ms)	local-accumulate (ms)	Counters correct? (Y/N)
1	300.11	95.23	40.11	Y*
2	297.51	90.42	33.25	Y*
3	300.42	94.99	37.79	Y*

TABLE 2 — Effect of Number of Threads

Threads	adjacent (ms)	padded (ms)	Slowdown adjacent / padded
1	81.18	83.92	**0.97×**
2	123.32	86.64	**1.42×**
4	302.98	169.94	**1.78×**
8	636.81	389.44	**1.64×**

Cache-line size (bytes): 64

Environment.ProcessorCount: 4

DISCUSSION ANSWERS

1. Why is the program with adjacent counters correct but slower, although no two threads ever write to the same variable?

Although the threads write to different variables (e.g., adjacent[0], adjacent[1]) and there is no logical data sharing, these variables are only 8 bytes apart in memory. This means they fit into the same 64-byte cache line. When Thread 0 updates its value, the CPU must invalidate the entire cache line in all other cores. When Thread 1 tries to update its value, it must fetch the latest line from memory. This causes the cache line to "bounce" between the cores, which significantly slows down execution (false sharing).

2. Why does padding fix the problem, and what does it cost? Why was the effect absent or small with 1 thread?

Padding fixes the problem by increasing the array stride to 8 longs (64 bytes). Now, each

thread's counter resides on its own separate cache line. When one core writes, the other cores' lines are not invalidated, eliminating the bouncing effect. **Cost:** It wastes memory (64 bytes are used per counter instead of 8). The effect is absent with 1 thread because there is only one core and one cache, so there is no other core to invalidate the cache line.

3. Which of the three versions is fastest? Explain why local accumulation is the best remedy in practice.

The local-accumulate version is the fastest (e.g., ~34 ms vs. ~170 ms for padded at 4 threads). It is fastest because it maintains its count in a local variable (CPU register) and does not touch shared memory inside the loop; it writes to the array only once at the end. Padding is still useful when data must be updated directly in shared memory structures (such as global arrays) rather than local variables.

4. How did the slowdown change as the number of threads grew, and what would you expect if the threads ran on a single core?

With 1 thread, adjacent and padded versions took the same amount of time (0.97x). As the number of threads increased (2, 4, 8), the slowdown of the adjacent version grew significantly (1.42x, 1.78x) due to increased cache contention. If all threads ran on a single core, false sharing would completely disappear because there is only one L1 cache, meaning no cross-core invalidation would occur.

5. How would you recognize false sharing in a real program, where the counters are not so obvious?

The biggest symptom is that the program becomes slower as you increase the number of threads (resulting in a negative speedup), even though the work is independent. To detect it, you can use performance profilers (such as Intel VTune, Visual Studio Concurrency Visualizer, or perf on Linux) to identify high cache-miss rates and pinpoint false sharing hotspots.

LAB TASK 04

Output

```
workload=uniform n=20000 degree=4 serial=271.3 (all times in ms)
[static      ] wall=   92.0 busy_max=   91.2 busy_min=   80.6 imbalance= 1.13 correct=YES
[static,1    ] wall=   75.4 busy_max=   74.1 busy_min=   66.8 imbalance= 1.11 correct=YES
[static,64   ] wall=   73.2 busy_max=   72.9 busy_min=   68.7 imbalance= 1.06 correct=YES
[dynamic,1   ] wall=   76.6 busy_max=   76.2 busy_min=   69.6 imbalance= 1.09 correct=YES
[dynamic,64  ] wall=   74.7 busy_max=   74.3 busy_min=   68.1 imbalance= 1.09 correct=YES
[guided,1    ] wall=   71.4 busy_max=   70.7 busy_min=   69.1 imbalance= 1.02 correct=YES
[guided,64   ] wall=   82.3 busy_max=   81.7 busy_min=   80.9 imbalance= 1.01 correct=YES
[Parallel.For] wall=   71.8 busy_max= n/a  busy_min= n/a  imbalance= n/a correct=YES
PS E:\Lab-07\Task2-SchedulingBench> cd..
```

Results

Summary of the two workloads (4 threads)

Policy	Uneven: wall (ms)	Uneven: imbalance	Uniform: wall (ms)	Uniform: imbalance	Verdict
static	109.5	5.95	92.0	1.13	Worst for Uneven, Moderate for Uniform
static,1	80.9	1.05	75.4	1.11	Excellent balance for Uneven
static,64	82.1	1.09	73.2	1.06	Good overall performer
dynamic,1	84.9	1.23	76.6	1.09	Good, but slight atomic overhead
dynamic,64	**74.9**	1.02	74.7	1.09	**Best for Uneven workload**
guided,1	107.4	1.01	**71.4**	1.02	**Best for Uniform workload**
guided,64	84.0	1.06	82.3	1.01	Good balance
Parallel.For	94.4	n/a	71.8	n/a	Best overall safe default

Written Summary (6 to 8 sentences)

On the uneven workload, the plain static scheduling performed the worst (109.5 ms) because the last thread received all the heavy iterations, resulting in a massive imbalance of 5.95. The dynamic,64 policy was the fastest (74.9 ms) on this workload because it allowed threads to claim chunks on demand, balancing the load with minimal atomic overhead. In contrast, on the uniform workload, the differences between policies shrank significantly. The simple, low-overhead policies like guided,1 (71.4 ms) and static,64 (73.2

ms) were the best, as dynamic chunk claiming only added unnecessary scheduling overhead. The Parallel.For library construct performed consistently well in both workloads, acting as a safe default that adapts dynamically. Therefore, the general rule of thumb is to use static scheduling with small chunks for uniform work to minimize overhead, and dynamic or guided scheduling for uneven workloads to ensure proper load balancing.

4.6 Discussion Answers

1. Which policy suited the uneven workload best on your machine, and which suited the uniform workload best? Support each answer with numbers from your table.
For the uneven workload, dynamic,64 suited best with a wall time of **74.9 ms** (compared to plain static at 109.5 ms). For the uniform workload, guided,1 suited best with a wall time of **71.4 ms**, closely followed by Parallel.For (71.8 ms) and static,64 (73.2 ms).

2. Why can static be the best or one of the best choices for the uniform workload, even though it was the worst for the uneven one?
In a uniform workload, every iteration takes the exact same amount of time. Therefore, if you divide the iterations evenly (static), every thread gets an equal amount of work, and they all finish at roughly the same time. There is no need for threads to dynamically claim work. This makes static incredibly efficient and fast because it avoids the overhead of atomic counters or locks. In an uneven workload, static fails because the chunks have varying costs, causing severe load imbalance.

3. If you did not know in advance how uneven the iterations are, which policy would you choose as a safe default, and why?
If the workload's unevenness is unknown, Parallel.For (or dynamic / guided) is the safest default. Dynamic and guided policies automatically balance the load by letting faster threads claim more chunks, which prevents one thread from becoming a severe bottleneck. The library Parallel.For does this adaptive chunking internally, making it safe and highly optimized for both uniform and uneven tasks.

4. What would change in your recommendation if each iteration were very cheap (a few nanoseconds), so that claiming a chunk costs as much as the work itself?
If iterations were extremely cheap, **static scheduling** would become the clear winner for almost any workload. This is because the overhead of claiming chunks dynamically (which requires atomic operations or lock acquisitions) would completely overshadow the

actual computation time. In that case, the cost of load balancing (dynamic/guided) would ruin performance.

5. Which of the hand-made policies resembles what Parallel.For does by itself, and what does that suggest about using the library default?

The dynamic and guided policies most closely resemble what Parallel.For does by itself. Parallel.For uses a dynamic, work-stealing task scheduler under the hood. This suggests that using the library default (Parallel.For) is highly recommended for production code; it provides excellent load balancing but is heavily optimized to minimize scheduling overhead, giving you the best of both worlds.

23436